Microsoft SQL Server

Ətraflı Təhsil Materialları

Prosedurlar, ACID Prinsipləri, Transaksiyalar, Rollback,
Sessiyalar, Connection Pooling və İzolyasiya Səviyyələri

Ətraflı Nəzəri və Praktik Bələdçi

Mündəricat

1. Stored Procedures (Saxlanmış Prosedurlar)
    - 1.1. Prosedur Əsasları
    - 1.2. Parametrlər və İstifadə Növləri
    - 1.3. Dinamik SQL və sp_executesql
    - 1.4. Error Handling və Debugging
    - 1.5. Performans və Optimization
2. ACID Prinsipləri
    - 2.1. Atomicity (Bölünməzlik)
    - 2.2. Consistency (Ardıcıllıq)
    - 2.3. Isolation (İzolyasiya)
    - 2.4. Durability (Davamlılıq)
    - 2.5. Real Dünya Ssenariləri
3. Transaksiyalar
    - 3.1. Transaksiya Növləri
    - 3.2. Transaksiya İdarəetməsi
    - 3.3. Nested Transactions
    - 3.4. Distributed Transactions
    - 3.5. Transaction Log
4. Rollback və Savepoint
    - 4.1. ROLLBACK Mexanizmi
    - 4.2. Savepoint Strategiyaları
    - 4.3. Partial Rollback
    - 4.4. Error Recovery Patterns
5. Sessiyalar və Connection Management
    - 5.1. Session Lifecycle

5.2. Session Configuration

5.3. Blocking və Deadlocks

5.4. Session Monitoring

6. Connection Pooling

6.1. Pooling Mexanizmi

6.2. Konfigurasiya və Tuning

6.3. Performance Monitoring

6.4. Best Practices

7. Isolation Levels

7.1. Read Uncommitted

7.2. Read Committed

7.3. Repeatable Read

7.4. Serializable

7.5. Snapshot Isolation

7.6. Lock Types və Escalation

8. Advanced Topics

8.1. Lock Management

8.2. Deadlock Prevention

8.3. Performance Tuning

8.4. Monitoring və Troubleshooting

1. Stored Procedures (Saxlanmış Prosedurlar)

1.1. Prosedur Əsasları

Stored Procedure - SQL Server-də saxlanılan, kompilyasiya olunan və təkrar istifadə edilə bilən SQL əmrlərinin toplusudur. Prosedurlar verilənlər bazası səviyyəsində biznes məntiqini yerləşdirmək, təhlükəsizliyi artırmaq və performansı optimallaşdırmaq üçün güclü vasitədir.

Stored Procedure-lərin Arxitekturası:

SQL Server stored procedure-ləri ilk dəfə icra edildikdə kompilyasiya olunur və execution plan procedure cache-də saxlanılır. Bu, sonrakı çağırışların daha sürətli olmasını təmin edir. Prosedurlar precompiled olduğu üçün, hər dəfə parse və optimize edilməsinə ehtiyac qalmır.

Əsas Üstünlüklər:

- Performans: Execution plan cache-də saxlanılır, təkrar kompilyasiya tələb olunmur.Network trafiki azalır çünki yalnız prosedur adı göndərilir.
- Təhlükəsizlik: SQL Injection hücumlarından tam qorunma. İstifadəçilərə prosedursəviyyəsində icazələr verilə bilər, cədvəllərə birbaşa giriş məhdudlaşdırıla bilər.
- Kodun təkrar istifadəsi: Mərkəzləşdirilmiş kod idarəetməsi. Bir dəfə yaz, hər yerdənçağır prinsipi.
- Encapsulation: Biznes məntiqinin mərkəzləşdirilməsi və mürəkkəbliyin gizlədilməsi.
- Versiyalaşdırma: Kodun versiyalaşdırılması və deployment idarəetməsi asanlaşır.
- Transaction İdarəetməsi: Kompleks transaction-ların idarə edilməsi daha asandır.

Sadə Prosedur Yaratma:

```
-- Ən sadə prosedur
CREATE PROCEDURE GetAllCustomers
AS
BEGIN
    SET NOCOUNT ON;  -- Rows affected mesajını söndür

    SELECT
        CustomerID,
        CustomerName,
        Email,
        Phone,
        City,
        Country,
        CreatedDate
    FROM Customers
    ORDER BY CustomerName;
END;
GO

-- Proseduru çağırmaq
EXEC GetAllCustomers;
-- və ya
EXECUTE GetAllCustomers;
```

1.2. Parametrlər və İstifadə Növləri

Input Parametrləri:

```sql
-- Input parametrli prosedur
CREATE PROCEDURE GetCustomerOrders
    @CustomerID INT,                     -- Məcburi parametr
    @StartDate DATE = NULL,             -- Optional parametr (default NULL)
    @EndDate DATE = NULL,
    @MinAmount DECIMAL(18,2) = 0,       -- Default dəyərli parametr
    @Status NVARCHAR(20) = 'All'        -- Default status
AS
BEGIN
    SET NOCOUNT ON;

    -- Parametr validasiyası
    IF @CustomerID IS NULL OR @CustomerID <= 0
    BEGIN
        RAISERROR('CustomerID məcburidir və müsbət olmalıdır!', 16, 1);
        RETURN;
    END

    -- Tarix parametrlərini təyin et
    SET @StartDate = ISNULL(@StartDate, '2000-01-01');
    SET @EndDate = ISNULL(@EndDate, GETDATE());

    -- Əsas sorğu
SELECT
        o.OrderID,
        o.OrderDate,
        o.TotalAmount,
        o.Status,
od.ProductID,
od.Quantity,
od.UnitPrice     FROM
Orders o
    INNER JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE
        o.CustomerID = @CustomerID         AND
o.OrderDate BETWEEN @StartDate AND @EndDate
        AND o.TotalAmount >= @MinAmount
        AND (@Status = 'All' OR o.Status = @Status)
    ORDER BY o.OrderDate DESC;
END;
GO

-- İstifadə nümunələri:
EXEC GetCustomerOrders @CustomerID = 1001;
EXEC GetCustomerOrders @CustomerID = 1001, @StartDate = '2024-01-01';
EXEC GetCustomerOrders @CustomerID = 1001, @Status = 'Completed', @MinAmount = 500;
```

Output Parametrləri:

```sql
-- OUTPUT parametrləri ilə prosedur
CREATE PROCEDURE CreateOrder
    @CustomerID INT,
    @ProductID INT,
    @Quantity INT,
    @OrderID INT OUTPUT,              -- OUTPUT parametr
    @TotalAmount DECIMAL(18,2) OUTPUT,
    @ErrorMessage NVARCHAR(500) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
```

```sql
    SET @ErrorMessage = NULL;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Məhsul qiymətini tap
        DECLARE @UnitPrice DECIMAL(18,2);
        SELECT @UnitPrice = Price
        FROM Products
        WHERE ProductID = @ProductID;

        IF @UnitPrice IS NULL
        BEGIN
            SET @ErrorMessage = 'Məhsul tapılmadı!';
            ROLLBACK;
            RETURN -1;
        END

        -- Toplam məbləği hesabla
        SET @TotalAmount = @UnitPrice * @Quantity;

        -- Sifariş yarat
        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
        VALUES (@CustomerID, GETDATE(), @TotalAmount, 'Pending');

        -- Yeni OrderID-ni əldə et
        SET @OrderID = SCOPE_IDENTITY();

        -- Sifariş detallarını əlavə et
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        VALUES (@OrderID, @ProductID, @Quantity, @UnitPrice);

        -- Stock-u yenilə
        UPDATE Products
        SET StockQuantity = StockQuantity - @Quantity
        WHERE ProductID = @ProductID;

        COMMIT TRANSACTION;
        RETURN 0;  -- Uğurlu

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        SET @ErrorMessage = ERROR_MESSAGE();
```

```
        SET @OrderID = -1;
        RETURN -1;  -- Uğursuz
    END CATCH
END;
GO

-- İstifadə:
DECLARE @NewOrderID INT;
DECLARE @Amount DECIMAL(18,2);
DECLARE @Error NVARCHAR(500);
DECLARE @Result INT;

EXEC @Result = CreateOrder
    @CustomerID = 1001,
    @ProductID = 5,
    @Quantity = 3,
    @OrderID = @NewOrderID OUTPUT,
    @TotalAmount = @Amount OUTPUT,
    @ErrorMessage = @Error OUTPUT;

IF @Result = 0
    PRINT 'Uğurlu! Order ID: ' + CAST(@NewOrderID AS VARCHAR) +
          ', Məbləğ: ' + CAST(@Amount AS VARCHAR);
ELSE
    PRINT 'Xəta: ' + @Error;
```

Table-Valued Parameters (TVP):

Table-Valued Parameters SQL Server-də çox güclü xüsusiyyətdir və bir prosedura bütöv cədvəl göndərməyə imkan verir. Bu, bulk insert əməliyyatları üçün çox effektivdir.

```
-- 1. User-Defined Table Type yarat
CREATE TYPE dbo.OrderItemType AS TABLE
(
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(18,2) NOT NULL,
    PRIMARY KEY (ProductID)
);
GO

-- 2. TVP istifadə edən prosedur
CREATE PROCEDURE CreateBulkOrder
    @CustomerID INT,
    @OrderItems dbo.OrderItemType READONLY,  -- Table parameter (READONLY!)
    @OrderID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Toplam məbləği hesabla
        DECLARE @TotalAmount DECIMAL(18,2);
        SELECT @TotalAmount = SUM(Quantity * UnitPrice)
        FROM @OrderItems;

        -- Order yarat
        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
        VALUES (@CustomerID, GETDATE(), @TotalAmount, 'Pending');

        SET @OrderID = SCOPE_IDENTITY();

        -- Bütün items-i bir sorğuda daxil et
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        SELECT @OrderID, ProductID, Quantity, UnitPrice
        FROM @OrderItems;

        -- Stock-ları yenilə
        UPDATE p
        SET p.StockQuantity = p.StockQuantity - oi.Quantity
        FROM Products p
        INNER JOIN @OrderItems oi ON p.ProductID = oi.ProductID;

        COMMIT TRANSACTION;

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

-- 3. İstifadə:
```

```
DECLARE @Items dbo.OrderItemType;

-- Items-i əlavə et
INSERT INTO @Items (ProductID, Quantity, UnitPrice)
VALUES
    (101, 5, 29.99),
    (102, 3, 49.99),
    (103, 2, 99.99);

-- Proseduru çağır
DECLARE @NewOrderID INT;
EXEC CreateBulkOrder
    @CustomerID = 1001,
    @OrderItems = @Items,
    @OrderID = @NewOrderID OUTPUT;

PRINT 'Order yaradıldı: ' + CAST(@NewOrderID AS VARCHAR);
```

1.3. Dinamik SQL və sp_executesql

Dinamik SQL runtime zamanı SQL əmrlərini qurmağa və icra etməyə imkan verir. Bu, məsələn, sütun adlarının və ya cədvəl adlarının parametr olaraq verilməsi lazım olduqda istifadə olunur. Ancaq dinamik SQL-in təhlükəsizlik riskləri var və ehtiyatla istifadə edilməlidir.

```
-- Dinamik SQL - SƏHV yanaşma (SQL Injection riski!)
CREATE PROCEDURE SearchProducts_Unsafe
    @SearchColumn NVARCHAR(50),
    @SearchValue NVARCHAR(100)
AS
BEGIN
    DECLARE @SQL NVARCHAR(MAX);
        -- TEHLÜKƏLİ! İstifadə
etməyin!
    SET @SQL = 'SELECT * FROM Products WHERE ' + @SearchColumn +
               ' LIKE ''%' + @SearchValue + '%''';
        EXEC(@SQL);  -- SQL Injection
risqi!
END;
GO

-- Dinamik SQL - DÜZGÜN yanaşma (sp_executesql)
CREATE PROCEDURE SearchProducts_Safe
    @SearchColumn NVARCHAR(50),
    @SearchValue NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    -- Sütun adını validate et (whitelist)
    IF @SearchColumn NOT IN ('ProductName', 'Description', 'Category')
    BEGIN           RAISERROR('Yalnız ProductName, Description və ya Category üzrə
axtarış mümkü ndür!', 16, 1);
```

```
        RETURN;
    END

    DECLARE @SQL NVARCHAR(MAX);
    DECLARE @Params NVARCHAR(MAX);

    -- Parametrləşdirilmiş dinamik SQL
    SET @SQL = N'SELECT ProductID, ProductName, Price, StockQuantity
                 FROM Products
                 WHERE ' + QUOTENAME(@SearchColumn) + N' LIKE @SearchPattern';

    SET @Params = N'@SearchPattern NVARCHAR(100)';

    -- sp_executesql istifadə et (parametrləşdirilmiş və təhlükəsiz)
EXEC sp_executesql @SQL, @Params, @SearchPattern = @SearchValue;
END;
GO

-- Dynamic sorting nümunəsi
CREATE PROCEDURE GetProducts
    @SortColumn NVARCHAR(50) = 'ProductName',
    @SortOrder NVARCHAR(4) = 'ASC',
    @PageNumber INT = 1,
    @PageSize INT = 20
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate inputs
    IF @SortColumn NOT IN ('ProductName', 'Price', 'StockQuantity', 'CreatedDate')
        SET @SortColumn = 'ProductName';

    IF @SortOrder NOT IN ('ASC', 'DESC')
        SET @SortOrder = 'ASC';

    DECLARE @SQL NVARCHAR(MAX);
    DECLARE @Params NVARCHAR(MAX);

    SET @SQL = N'
        SELECT
            ProductID,
            ProductName,
            Price,
            StockQuantity,
            CreatedDate
        FROM Products
        ORDER BY ' + QUOTENAME(@SortColumn) + ' ' + @SortOrder + N'
        OFFSET @Skip ROWS
        FETCH NEXT @Take ROWS ONLY';

    SET @Params = N'@Skip INT, @Take INT';

    EXEC sp_executesql @SQL, @Params,
        @Skip = (@PageNumber - 1) * @PageSize,
        @Take = @PageSize;
END;
GO
```

1.4. Error Handling və Debugging

```
-- Kompleks Error Handling nümunəsi
CREATE PROCEDURE ProcessComplexTransaction
    @CustomerID INT,
    @Items dbo.OrderItemType READONLY,
    @DiscountCode NVARCHAR(50) = NULL,
    @OrderID INT OUTPUT,
    @TotalAmount DECIMAL(18,2) OUTPUT,
    @DiscountAmount DECIMAL(18,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;  -- Kritik xətalarda avtomatik rollback

    DECLARE @ErrorNumber INT;
    DECLARE @ErrorSeverity INT;
    DECLARE @ErrorState INT;
    DECLARE @ErrorMessage NVARCHAR(4000);
    DECLARE @ErrorLine INT;
    DECLARE @ErrorProc NVARCHAR(128);

    BEGIN TRY
        BEGIN TRANSACTION ProcessOrder;

        -- 1. Müştəri validasiyası
        IF NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = @CustomerID)
        BEGIN
            THROW 50001, 'Müştəri tapılmadı!', 1;
        END

        -- 2. Items validasiyası
        IF NOT EXISTS (SELECT 1 FROM @Items)
        BEGIN
            THROW 50002, 'Ən azı bir məhsul seçilməlidir!', 1;
        END

        -- 3. Stock yoxlanışı
        IF EXISTS (
            SELECT 1
            FROM @Items i
            INNER JOIN Products p ON i.ProductID = p.ProductID
            WHERE p.StockQuantity < i.Quantity
        )
        BEGIN
            DECLARE @OutOfStockProducts NVARCHAR(MAX);
            SELECT @OutOfStockProducts = STRING_AGG(ProductName, ', ')
            FROM Products p
            INNER JOIN @Items i ON p.ProductID = i.ProductID
            WHERE p.StockQuantity < i.Quantity;

            THROW 50003, 'Stock kifayət deyil: ' + @OutOfStockProducts, 1;
        END

        -- 4. Endirim kodu yoxla
        SET @DiscountAmount = 0;
        IF @DiscountCode IS NOT NULL
        BEGIN
            DECLARE @DiscountPercent DECIMAL(5,2);
```

```
            SELECT @DiscountPercent = DiscountPercent
            FROM DiscountCodes
            WHERE Code = @DiscountCode
              AND IsActive = 1
              AND ValidFrom <= GETDATE()
              AND ValidTo >= GETDATE();

            IF @DiscountPercent IS NULL
            BEGIN
                -- Endirim kodu etibarsızdır, amma əməliyyatı davam etdir
                PRINT 'Xəbərdarlıq: Endirim kodu etibarsızdır və tətbiq olunmadı.';
            END
            ELSE
            BEGIN
                SET @DiscountAmount = (@TotalAmount * @DiscountPercent) / 100;
            END
        END

        -- 5. Məbləği hesabla
        SELECT @TotalAmount = SUM(Quantity * UnitPrice)
        FROM @Items;

        -- 6. Order yarat         INSERT INTO Orders (CustomerID, OrderDate,
TotalAmount, DiscountAmount, Fina lAmount, Status)         VALUES (
            @CustomerID,
            GETDATE(),
            @TotalAmount,
            @DiscountAmount,
            @TotalAmount - @DiscountAmount,
            'Pending'
        );

        SET @OrderID = SCOPE_IDENTITY();

        -- 7. Order details
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        SELECT @OrderID, ProductID, Quantity, UnitPrice
        FROM @Items;

        -- 8. Stock-u azalt
        UPDATE p         SET p.StockQuantity =
p.StockQuantity - i.Quantity,
            p.LastModifiedDate = GETDATE()
FROM Products p
        INNER JOIN @Items i ON p.ProductID = i.ProductID;

        -- 9. Log qeyd et
        INSERT INTO TransactionLog (OrderID, Action, Timestamp, UserID)
VALUES (@OrderID, 'Order Created', GETDATE(), SYSTEM_USER);

        -- Uğurlu olarsa commit et
        COMMIT TRANSACTION ProcessOrder;

        PRINT 'Sifariş uğurla yaradıldı! Order ID: ' + CAST(@OrderID AS VARCHAR);
        RETURN 0;
```

```
    END TRY
    BEGIN CATCH
        -- Transaksiya açıqdırsa, rollback et
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION ProcessOrder;

        -- Xəta məlumatlarını əldə et
        SELECT
            @ErrorNumber = ERROR_NUMBER(),
            @ErrorSeverity = ERROR_SEVERITY(),
            @ErrorState = ERROR_STATE(),
            @ErrorMessage = ERROR_MESSAGE(),
            @ErrorLine = ERROR_LINE(),
            @ErrorProc = ERROR_PROCEDURE();

        -- Xətanı log-a yaz
        INSERT INTO ErrorLog (
            ErrorNumber,
            ErrorSeverity,
            ErrorState,
            ErrorMessage,
            ErrorLine,
            ErrorProcedure,
            ErrorTime,
            UserName
        )
        VALUES (
            @ErrorNumber,
            @ErrorSeverity,
            @ErrorState,
            @ErrorMessage,
            @ErrorLine,
            @ErrorProc,
            GETDATE(),
            SYSTEM_USER
        );

        -- Ətraflı xəta mesajı
        SET @ErrorMessage = 'Xəta baş verdi!' + CHAR(13) + CHAR(10) +
                            'Prosedur: ' + ISNULL(@ErrorProc, 'N/A') + CHAR(13) + CHA
R(10) +
                            'Sətir: ' + CAST(@ErrorLine AS VARCHAR) + CHAR(13) + CHAR
(10) +                                  'Mesaj: ' +
@ErrorMessage;

        -- Xətanı yenidən qaldır
        RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);

        RETURN -1;
    END CATCH
END;
GO
```

Debugging və Monitoring:

```
-- Prosedur performansını monitor et
-- 1. Execution statistics
SELECT
    p.name AS ProcedureName,
    s.execution_count AS ExecutionCount,
    s.total_worker_time / 1000000 AS TotalCPU_Seconds,
    s.total_elapsed_time / 1000000 AS TotalElapsed_Seconds,
    s.total_logical_reads AS TotalLogicalReads,
    s.total_logical_writes AS TotalLogicalWrites,
    s.last_execution_time AS LastExecuted,
    s.cached_time AS CachedTime
FROM sys.procedures p
INNER JOIN sys.dm_exec_procedure_stats s ON p.object_id = s.object_id
WHERE p.name = 'ProcessComplexTransaction'
ORDER BY s.total_worker_time DESC;

-- 2. Prosedur definition-ı gör
EXEC sp_helptext 'ProcessComplexTransaction';

-- 3. Prosedur haqqında ümumi məlumat
EXEC sp_help 'ProcessComplexTransaction';

-- 4. Proseduru debug etmək üçün PRINT istifadə et
CREATE PROCEDURE DebugExample
    @CustomerID INT
AS
BEGIN
    PRINT 'Prosedur başladı: ' + CONVERT(VARCHAR, GETDATE(), 121);
    PRINT 'CustomerID: ' + CAST(@CustomerID AS VARCHAR);

    DECLARE @OrderCount INT;
    SELECT @OrderCount = COUNT(*) FROM Orders WHERE CustomerID = @CustomerID;
    PRINT 'Mövcud sifarişlər: ' + CAST(@OrderCount AS VARCHAR);
        --
əməliyyatlar...

    PRINT 'Prosedur başa çatdı: ' + CONVERT(VARCHAR, GETDATE(), 121);
END;
GO

-- 5. Execution plan-ı gör
SET SHOWPLAN_ALL ON;
EXEC GetCustomerOrders @CustomerID = 1001;
SET SHOWPLAN_ALL OFF;

-- 6. Actual execution statistics
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
EXEC ProcessComplexTransaction @CustomerID = 1001, @Items = @TestItems;
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

1.5. Performans və Optimization

Prosedur Recompilation:

SQL Server bəzi hallarda prosedurları recompile edir. Bu, plan cache-dən istifadə etmək əvəzinə yeni execution plan yaratmaq deməkdir. Recompilation performansa təsir göstərə bilər.

```
-- Recompilation səbəbləri və həllər
```

```sql
-- 1. Parameter Sniffing problemi
CREATE PROCEDURE GetOrdersByAmount
    @MinAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    -- Problem: İlk dəfə çağırılanda istifadə olunan parametr üzrə plan yaradılır
    SELECT OrderID, CustomerID, TotalAmount, OrderDate
    FROM Orders
    WHERE TotalAmount >= @MinAmount;

    -- Əgər sonra çox fərqli parametr gəlsə, plan optimal olmaya bilər
END;
GO

-- Həll 1: OPTION (RECOMPILE) - hər dəfə yeni plan yarat
CREATE PROCEDURE GetOrdersByAmount_Fix1
    @MinAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT OrderID, CustomerID, TotalAmount, OrderDate
    FROM Orders
    WHERE TotalAmount >= @MinAmount
    OPTION (RECOMPILE);  -- Hər dəfə recompile et
END;
GO

-- Həll 2: OPTION (OPTIMIZE FOR) - müəyyən dəyər üzrə optimize et
CREATE PROCEDURE GetOrdersByAmount_Fix2
    @MinAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT OrderID, CustomerID, TotalAmount, OrderDate
    FROM Orders
    WHERE TotalAmount >= @MinAmount
    OPTION (OPTIMIZE FOR (@MinAmount = 1000));  -- 1000 dəyəri üzrə optimize et
END;
GO

-- Həll 3: Local variable trick (parameter sniffing-dən qaçmaq)
CREATE PROCEDURE GetOrdersByAmount_Fix3
    @MinAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @LocalMinAmount DECIMAL(18,2) = @MinAmount;
```

```
    -- Local variable ilə parameter sniffing olmur
    SELECT OrderID, CustomerID, TotalAmount, OrderDate
    FROM Orders
    WHERE TotalAmount >= @LocalMinAmount;
END;
GO

-- 2. Temp table əvəzinə Table variable (kiçik data üçün)
CREATE PROCEDURE ProcessSmallBatch
AS
BEGIN
    -- Kiçik data (< 100 rows) üçün table variable daha yaxşıdır
    DECLARE @TempOrders TABLE (
        OrderID INT,
        CustomerID INT,
        TotalAmount DECIMAL(18,2)
    );

    INSERT INTO @TempOrders
    SELECT TOP 50 OrderID, CustomerID, TotalAmount
    FROM Orders
    WHERE Status = 'Pending';
        --
əməliyyatlar...
END;
GO

-- Böyük data (> 100 rows) üçün temp table
CREATE PROCEDURE ProcessLargeBatch
AS
BEGIN
    CREATE TABLE #TempOrders (
        OrderID INT PRIMARY KEY,
        CustomerID INT,
        TotalAmount DECIMAL(18,2),         INDEX
IX_Customer (CustomerID)  -- İndeks əlavə et!
    );

    INSERT INTO #TempOrders
    SELECT OrderID, CustomerID, TotalAmount
    FROM Orders
    WHERE Status = 'Pending';

    -- Statistikaları yenilə (böyük data üçün)
    UPDATE STATISTICS #TempOrders;
        --
əməliyyatlar...

    DROP TABLE #TempOrders;
END;
GO

-- 3. SET NOCOUNT ON istifadə et (performans)
CREATE PROCEDURE FastProcedure
AS
BEGIN
    SET NOCOUNT ON;  -- "XX rows affected" mesajını söndür
```

```sql
    -- Bu, network trafiki azaldır və performansı artırır
    SELECT * FROM Customers;
END;
GO

-- 4. WHERE 1=1 pattern (dinamik WHERE üçün)
CREATE PROCEDURE SearchOrders
    @CustomerID INT = NULL,
    @Status NVARCHAR(20) = NULL,
    @MinAmount DECIMAL(18,2) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT OrderID, CustomerID, TotalAmount, Status
    FROM Orders
    WHERE 1=1  -- Bu, dinamik WHERE şərtləri əlavə etməyi asanlaşdırır
        AND (@CustomerID IS NULL OR CustomerID = @CustomerID)
        AND (@Status IS NULL OR Status = @Status)
        AND (@MinAmount IS NULL OR TotalAmount >= @MinAmount);
END;
GO
```

2. ACID Prinsipləri

ACID prinsipləri verilənlər bazası transaksiyalarının etibarlılığını təmin edən dörd əsas prinsipdir. Bu prinsiplər məlumatların bütövlüyünü və ardıcıllığını qoruyur. Hər bir prinsip kritik əhəmiyyət daşıyır və birlikdə məlumat bütövlüyünü təmin edir.

2.1. Atomicity (Bölünməzlik)

Atomicity prinsipi transaksiyanın 'hamısı və ya heç biri' prinsipi ilə işləməsini təmin edir. Transaksiya daxilindəki bütün əməliyyatlar ya uğurla icra olunur, ya da heç biri icra olunmur. Hissə-hissə icra mümkün deyil. Bu, məlumat tutarlılığını qoruyur.

```
-- Atomicity nümunəsi: Bank transferi
CREATE PROCEDURE TransferMoney
    @FromAccount INT,
    @ToAccount INT,
    @Amount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;  -- Transaksiya başla

        -- Əməliyyat 1: Göndərən hesabdan pulu çıxart
        UPDATE BankAccounts
        SET Balance = Balance - @Amount
        WHERE AccountID = @FromAccount;

        -- Əməliyyat 2: Balansın mənfi olmamasını yoxla
        DECLARE @FromBalance DECIMAL(18,2);
        SELECT @FromBalance = Balance
        FROM BankAccounts
        WHERE AccountID = @FromAccount;

        IF @FromBalance < 0
        BEGIN             -- Kifayət qədər balans
yoxdur - GERİ AL!
            THROW 50000, 'Kifayət qədər balans yoxdur!', 1;
        END

        -- Əməliyyat 3: Qəbul edən hesaba pulu əlavə et
        UPDATE BankAccounts
        SET Balance = Balance + @Amount
        WHERE AccountID = @ToAccount;

        -- Əməliyyat 4: Transfer qeydini yarat
        INSERT INTO TransferLog (FromAccount, ToAccount, Amount, TransferDate)
        VALUES (@FromAccount, @ToAccount, @Amount, GETDATE());

        -- HAMISI uğurlu olarsa - COMMIT
        COMMIT TRANSACTION;
        PRINT 'Transfer uğurla başa çatdı!';

    END TRY
    BEGIN CATCH

        -- HƏR HANSI xəta olarsa - ROLLBACK (heç bir dəyişiklik qalmır!)
```

```
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        PRINT 'Transfer uğursuz oldu: ' + ERROR_MESSAGE();
        PRINT 'Bütün dəyişikliklər geri alındı! (Atomicity)';
        THROW;
    END CATCH
END;
GO

-- Test: Uğurlu transfer
EXEC TransferMoney @FromAccount = 1, @ToAccount = 2, @Amount = 100;

-- Test: Uğursuz transfer (kifayət qədər pul yoxdur)
EXEC TransferMoney @FromAccount = 1, @ToAccount = 2, @Amount = 999999; -- Nəticə:
Heç bir dəyişiklik olmayacaq!
```

2.2. Consistency (Ardıcıllıq)

Consistency prinsipi verilənlər bazasının həmişə etibarlı (valid) vəziyyətdə olmasını təmin edir. Transaksiya başlamazdan əvvəl verilənlər bazası etibarlı vəziyyətdədir, transaksiyadan sonra da etibarlı vəziyyətdə olmalıdır. Bütün məhdudiyyətlər, trigger-lər və qaydalar saxlanılmalıdır.

```
-- Consistency nümunəsi: Constraints və Business Rules

-- 1. Database constraints (FK, CHECK və s.)
CREATE TABLE Orders_Consistent (
    OrderID INT PRIMARY KEY IDENTITY,
    CustomerID INT NOT NULL,
    OrderDate DATE NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(18,2) NOT NULL,
    Status NVARCHAR(20) NOT NULL,

    -- Foreign Key constraint - müştəri mövcud olmalıdır
    CONSTRAINT FK_Orders_Customer
        FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),

    -- Check constraint - məbləğ müsbət olmalıdır
    CONSTRAINT CK_Orders_Amount
        CHECK (TotalAmount > 0),

    -- Check constraint - status etibarlı olmalıdır
    CONSTRAINT CK_Orders_Status
        CHECK (Status IN ('Pending', 'Processing', 'Completed', 'Cancelled'))
);

-- 2. Trigger ilə consistency təmin etmək
CREATE TRIGGER trg_OrderDetails_StockCheck
ON OrderDetails
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Stock kifayət edərsə davam et, yoxsa rollback
```

```
    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN Products p ON i.ProductID = p.ProductID
        WHERE p.StockQuantity < i.Quantity
    )
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50000, 'Stock kifayət etmir! Consistency qorundu.', 1;
    END

    -- Stock-u avtomatik azalt
    UPDATE p
    SET p.StockQuantity = p.StockQuantity - i.Quantity
    FROM Products p
    INNER JOIN inserted i ON p.ProductID = i.ProductID;
END;
GO

-- 3. Application-level consistency prosedurda
CREATE PROCEDURE CreateOrder_Consistent
    @CustomerID INT,
    @ProductID INT,
    @Quantity INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Consistency yoxlaması 1: Müştəri aktiv olmalıdır
        DECLARE @IsActive BIT;
        SELECT @IsActive = IsActive FROM Customers WHERE CustomerID = @CustomerID;

        IF @IsActive = 0 OR @IsActive IS NULL
        BEGIN
            THROW 50001, 'Müştəri aktiv deyil və ya tapılmadı!', 1;
        END

        -- Consistency yoxlaması 2: Məhsul satışda olmalıdır
        DECLARE @IsForSale BIT;
        SELECT @IsForSale = IsForSale FROM Products WHERE ProductID = @ProductID;

        IF @IsForSale = 0 OR @IsForSale IS NULL
        BEGIN
            THROW 50002, 'Məhsul satışda deyil və ya tapılmadı!', 1;
        END

        -- Consistency yoxlaması 3: Müştərinin kredit limiti
        DECLARE @CreditLimit DECIMAL(18,2), @CurrentDebt DECIMAL(18,2), @OrderAmount
 DECIMAL(18,2);

        SELECT @CreditLimit = CreditLimit, @CurrentDebt = CurrentDebt
        FROM Customers WHERE CustomerID = @CustomerID;
                SELECT @OrderAmount = Price * @Quantity FROM Products WHERE ProductID
= @Pro ductID;
```

```
        IF (@CurrentDebt + @OrderAmount) > @CreditLimit
        BEGIN
            THROW 50003, 'Kredit limiti aşıldı!', 1;
        END

        -- Hamısı OK-dırsa, order yarat
        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
        VALUES (@CustomerID, GETDATE(), @OrderAmount, 'Pending');

        DECLARE @OrderID INT = SCOPE_IDENTITY();

        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        SELECT @OrderID, ProductID, @Quantity, Price
        FROM Products WHERE ProductID = @ProductID;

        -- Müştərinin borcunu yenilə
        UPDATE Customers
        SET CurrentDebt = CurrentDebt + @OrderAmount
        WHERE CustomerID = @CustomerID;

        COMMIT TRANSACTION;

        PRINT 'Sifariş yaradıldı! Bütün consistency qaydalarına əməl olundu.';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

2.3. Isolation (İzolyasiya)

Isolation prinsipi paralel transaksiyaların bir-birinə təsir etməməsini təmin edir. Hər transaksiya digər transaksiyalardan təcrid olunmuş şəkildə işləyir. İzolyasiya səviyyələri (Isolation Levels) bu təcridliyin dərəcəsini müəyyən edir və performans ilə məlumat tutarlılığı arasında balans yaradır.

```
-- Isolation nümunəsi: İki paralel transaksiya

-- Session 1: Müştəri məlumatlarını yeniləyir
BEGIN TRANSACTION;
    UPDATE Customers
    SET Email = 'newemail@example.com'
    WHERE CustomerID = 1001;

    -- 10 saniyə gözlə (test üçün)
    WAITFOR DELAY '00:00:10';

COMMIT;

-- Session 2: Eyni müştərini oxuyur (eyni vaxtda)
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  -- Default
SELECT Email FROM Customers WHERE CustomerID = 1001;
```

```
-- Nəticə: Session 2 gözləyəcək, Session 1 commit edənə qədər -- Bu Isolation
prinsipidir!

-- Fərqli isolation level ilə:
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  -- Dirty read
SELECT Email FROM Customers WHERE CustomerID = 1001;
-- Bu halda, Session 2 dərhal oxuyacaq (committed olmadan)

-- Real-world nümunə: Bilet satışı
CREATE PROCEDURE BookTicket
    @EventID INT,
    @CustomerID INT,
    @SeatNumber INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Yüksək isolation level - double booking-dən qorunmaq üçün
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Yeri yoxla və bloklayır (SERIALIZABLE səbəbindən)
        DECLARE @IsBooked BIT;
        SELECT @IsBooked = IsBooked
        FROM EventSeats WITH (HOLDLOCK)  -- Explicit lock
        WHERE EventID = @EventID AND SeatNumber = @SeatNumber;

        IF @IsBooked = 1
        BEGIN
            THROW 50000, 'Yer artıq bron edilib!', 1;
        END

        -- 2 paralel istifadəçi eyni yeri bron edə bilməz
        UPDATE EventSeats
        SET IsBooked = 1, CustomerID = @CustomerID, BookedDate = GETDATE()
        WHERE EventID = @EventID AND SeatNumber = @SeatNumber;

        INSERT INTO Bookings (EventID, CustomerID, SeatNumber, BookingDate)
        VALUES (@EventID, @CustomerID, @SeatNumber, GETDATE());

        COMMIT TRANSACTION;
        PRINT 'Bilet uğurla bron edildi!';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

2.4. Durability (Davamlılıq)

Durability prinsipi transaksiya uğurla başa çatdıqdan sonra (COMMIT), sistem nasazlığı olsa belə nəticələrin saxlanmasını təmin edir. COMMIT əmrindən sonra məlumat transaction log-a yazılır və disk-ə flush edilir. Elektrik kəsilməsi, server çökməsi və s. olsa belə, məlumat itirilmir.

```
-- Durability nümunəsi

-- 1. Transaksiya uğurla commit olur
BEGIN TRANSACTION;
    INSERT INTO CriticalData (DataValue, CreatedDate)
    VALUES ('Vacib məlumat', GETDATE());

    -- Log-a yazılır və disk-ə flush edilir
COMMIT TRANSACTION; -- Bu nöqtədən sonra, sistem çöksə belə,

məlumat saxlanılır!

-- 2. Durability test (CHECKPOINT)
CHECKPOINT;  -- Dirty pages-ləri disk-ə yazır

-- 3. Transaction log baxımı
-- Transaction log Durability-ni təmin edir
SELECT
    [Current LSN],
    [Transaction ID],
    Operation,
    [Transaction Name],
    [Begin Time]
FROM fn_dblog(NULL, NULL)
WHERE Operation IN ('LOP_BEGIN_XACT', 'LOP_COMMIT_XACT', 'LOP_ABORT_XACT')
ORDER BY [Begin Time] DESC;

-- 4. Kritik əməliyyat üçün forced write
CREATE PROCEDURE CriticalTransaction
    @Data NVARCHAR(MAX)
```

```
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Kritik məlumat
        INSERT INTO CriticalData (DataValue, CreatedDate)
        VALUES (@Data, GETDATE());

        -- Dərhal commit et və disk-ə yaz
        COMMIT TRANSACTION;

        -- Manual checkpoint (test üçün - production-da ehtiyac yoxdur)
        -- CHECKPOINT;

        PRINT 'Məlumat təhlükəsiz şəkildə saxlanıldı (Durability)!';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

-- 5. Database recovery model (Durability üçün vacib)
-- FULL recovery model - maksimum durability
ALTER DATABASE MyDatabase SET RECOVERY FULL;

-- Transaction log backup (Point-in-time recovery)
BACKUP LOG MyDatabase TO DISK = 'C:\Backup\MyDatabase_Log.trn';
```

2.5. Real Dünya Ssenariləri

```
-- Kompleks Real-World Scenario: E-Commerce Order Processing
CREATE PROCEDURE ProcessECommerceOrder
    @CustomerID INT,
    @OrderItems dbo.OrderItemType READONLY,
    @PaymentMethod NVARCHAR(50),
    @ShippingAddress NVARCHAR(500),
    @OrderID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @TotalAmount DECIMAL(18,2);
    DECLARE @InventoryReserved BIT = 0;
    DECLARE @PaymentProcessed BIT = 0;

    BEGIN TRY
        BEGIN TRANSACTION ECommOrder;  -- ATOMICITY başlayır

        -- ========== CONSISTENCY: Bütün məhdudiyyətləri yoxla ==========
```

```
        -- 1. Müştəri validasiyası
        IF NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = @CustomerID AND Is
Active = 1)
        BEGIN
            THROW 50001, 'Müştəri tapılmadı və ya aktiv deyil!', 1;
        END

        -- 2. Items validasiyası
        IF NOT EXISTS (SELECT 1 FROM @OrderItems)
        BEGIN
            THROW 50002, 'Sifariş boş ola bilməz!', 1;
        END

        -- 3. Bütün məhsulların mövcudluğunu yoxla
        IF EXISTS (
            SELECT 1 FROM @OrderItems i
            LEFT JOIN Products p ON i.ProductID = p.ProductID
            WHERE p.ProductID IS NULL OR p.IsActive = 0
        )
        BEGIN
            THROW 50003, 'Bəzi məhsullar mövcud deyil və ya aktiv deyil!', 1;
        END

        -- ========== ISOLATION: Kritik resursları bloklayır ==========

        -- Stock-u yoxla və reserve et (UPDLOCK - başqaları yaza bilməz)
        IF EXISTS (
            SELECT 1
            FROM @OrderItems i             INNER JOIN Products p WITH (UPDLOCK,
HOLDLOCK) ON i.ProductID = p.Produc tID             WHERE p.StockQuantity <
i.Quantity
        )
        BEGIN
            DECLARE @OutOfStock NVARCHAR(MAX);
            SELECT @OutOfStock = STRING_AGG(p.ProductName, ', ')
```

```
            FROM @OrderItems i
            INNER JOIN Products p ON i.ProductID = p.ProductID
            WHERE p.StockQuantity < i.Quantity;

            THROW 50004, 'Stock kifayət etmir: ' + @OutOfStock, 1;
        END

        SET @InventoryReserved = 1;

        -- Stock-u azalt (ISOLATION - başqa sessiyalar gözləyir)
        UPDATE p         SET p.StockQuantity =
p.StockQuantity - i.Quantity,
            p.ReservedQuantity = p.ReservedQuantity + i.Quantity
FROM Products p
        INNER JOIN @OrderItems i ON p.ProductID = i.ProductID;

        -- ========== DURABILITY: Məlumatı davamlı yaz ==========

        -- Toplam məbləği hesabla
        SELECT @TotalAmount = SUM(Quantity * UnitPrice)
        FROM @OrderItems;

        -- Order yarat (Log-a yazılır)
        INSERT INTO Orders (
            CustomerID,
            OrderDate,
            TotalAmount,
            Status,
            PaymentMethod,
            ShippingAddress
        )
        VALUES (
            @CustomerID,
            GETDATE(),
            @TotalAmount,
            'Pending',
            @PaymentMethod,
            @ShippingAddress
        );

        SET @OrderID = SCOPE_IDENTITY();

        -- Order details (DURABILITY - log-a yazılır)
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        SELECT @OrderID, ProductID, Quantity, UnitPrice
        FROM @OrderItems;

        -- ========== Payment Processing ==========

        -- External payment gateway-ə müraciət (kompensasiya lazım ola bilər)
        DECLARE @PaymentResult INT;
        EXEC @PaymentResult = ProcessPayment
            @OrderID = @OrderID,
            @Amount = @TotalAmount,
            @PaymentMethod = @PaymentMethod;

        IF @PaymentResult <> 0
        BEGIN
```

```
            -- Payment uğursuz - kompensasiya et
            UPDATE p             SET p.StockQuantity =
p.StockQuantity + i.Quantity,
                p.ReservedQuantity = p.ReservedQuantity - i.Quantity
FROM Products p
            INNER JOIN @OrderItems i ON p.ProductID = i.ProductID;

            THROW 50005, 'Ödəniş uğursuz oldu!', 1;
        END

        SET @PaymentProcessed = 1;

        -- Status-u yenilə
        UPDATE Orders
        SET Status = 'Paid', PaidDate = GETDATE()
        WHERE OrderID = @OrderID;

        -- Inventory-dən rezervasiyaları real stock-a çevir
        UPDATE p
        SET p.ReservedQuantity = p.ReservedQuantity - i.Quantity
        FROM Products p
        INNER JOIN @OrderItems i ON p.ProductID = i.ProductID;

        -- Audit log
        INSERT INTO OrderAuditLog (OrderID, Action, ActionDate, UserID)
        VALUES (@OrderID, 'Order Created and Paid', GETDATE(), SYSTEM_USER);

        -- ========== COMMIT: ACID prinsipləri təmin olundu ==========
        COMMIT TRANSACTION ECommOrder;

        PRINT 'Sifariş uğurla emal olundu! Order ID: ' + CAST(@OrderID AS VARCHAR);
        PRINT 'ACID prinsipləri tətbiq olundu:';
        PRINT '  • Atomicity: Bütün əməliyyatlar uğurla tamamlandı';
        PRINT '  • Consistency: Bütün qaydalar və məhdudiyyətlər saxlanıldı';
        PRINT '  • Isolation: Paralel əməliyyatlardan təcrid olundu';
PRINT '  • Durability: Məlumat davamlı şəkildə saxlanıldı';

    END TRY
    BEGIN CATCH
        -- ========== Rollback və Kompensasiya ==========
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION ECommOrder;

        -- Kompensasiya məntiqii (əgər external sistem dəyişdiyisə)
        IF @PaymentProcessed = 1
        BEGIN
            -- Payment refund et
            EXEC RefundPayment @OrderID = @OrderID;
        END

        -- Xətanı log-a yaz
        INSERT INTO ErrorLog (
            ErrorMessage,
            ErrorProcedure,
            ErrorLine,
            ErrorTime,
            OrderID
        )
```

```
        VALUES (
            ERROR_MESSAGE(),
            ERROR_PROCEDURE(),
            ERROR_LINE(),
            GETDATE(),
            @OrderID
        );

        THROW;
    END CATCH
END;
GO
```

Qeyd: ACID prinsipləri bir-biri ilə sıx əlaqəlidir və birlikdə işləyir. Atomicity olmadan Consistency təmin edilə bilməz. Isolation olmadan Consistency risk altındadır. Durability olmadan Atomicity və Consistency mənasız olur. Bütün dörd prinsip birlikdə məlumat bütövlüyünü və etibarlılığını təmin edir.

3. Transaksiyalar (Transactions)

3.1. Transaksiya Növləri

Transaksiya SQL Server-də bir və ya daha çox SQL əmrindən ibarət olan və bütövlükdə icra olunmalı olan iş vahididir. SQL Server üç növ transaksiya dəstəkləyir: Explicit (Açıq), Implicit (Gizli) və Autocommit (Avtomatik).

```
-- 1. EXPLICIT TRANSACTIONS (Ən çox istifadə olunan)
BEGIN TRANSACTION;  -- və ya BEGIN TRAN
    UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
    UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
COMMIT TRANSACTION;  -- və ya COMMIT

-- 2. NAMED TRANSACTIONS (Debugging üçün faydalıdır)
BEGIN TRANSACTION MoneyTransfer;
    DECLARE @Error INT = 0;

    UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
    SET @Error = @@ERROR;

    IF @Error <> 0
    BEGIN
        ROLLBACK TRANSACTION MoneyTransfer;
        PRINT 'Transfer ləğv edildi';
    END
    ELSE
    BEGIN
        UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
        COMMIT TRANSACTION MoneyTransfer;
        PRINT 'Transfer tamamlandı';
    END
GO

-- 3. IMPLICIT TRANSACTIONS
SET IMPLICIT_TRANSACTIONS ON;

-- Bu rejimdə hər SQL əmri avtomatik transaksiya başladır
UPDATE Products SET Price = 100 WHERE ProductID = 1;
-- COMMIT və ya ROLLBACK əl ilə edilməlidir
COMMIT;

SET IMPLICIT_TRANSACTIONS OFF;

-- 4. AUTOCOMMIT (Default rejim)
-- Hər SQL əmri avtomatik transaksiya kimi işləyir və avtomatik commit olunur
UPDATE Products SET Price = 100 WHERE ProductID = 1;
-- Avtomatik commit olunur

-- @@TRANCOUNT - Aktiv transaksiya səviyyəsini göstərir
SELECT @@TRANCOUNT AS ActiveTransactions;  -- 0 = transaksiya yoxdur

BEGIN TRANSACTION;
SELECT @@TRANCOUNT;  -- 1
    BEGIN TRANSACTION;
    SELECT @@TRANCOUNT;  -- 2
    COMMIT;
SELECT @@TRANCOUNT;  -- 1
```

```
COMMIT;
SELECT @@TRANCOUNT;  -- 0
```

3.2. Transaksiya İdarəetməsi

```
-- Kompleks Transaksiya İdarəetməsi Nümunəsi
CREATE PROCEDURE ComplexOrderProcessing
    @CustomerID INT,
    @Items dbo.OrderItemType READONLY
AS
BEGIN
    SET NOCOUNT ON;

    -- Transaksiya dəyişənləri
    DECLARE @TransactionStarted BIT = 0;
    DECLARE @SavePointName VARCHAR(50);
    DECLARE @OrderID INT;
    DECLARE @StepNumber INT = 0;

    BEGIN TRY
        -- Transaksiya başlat
        IF @@TRANCOUNT = 0
        BEGIN
            BEGIN TRANSACTION ComplexOrder;
            SET @TransactionStarted = 1;
        END

        -- Step 1: Müştəri validasiyası
        SET @StepNumber = 1;
        SET @SavePointName = 'AfterCustomerValidation';

        IF NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = @CustomerID AND Is
Active = 1)
        BEGIN
            THROW 50001, 'Müştəri tapılmadı və ya deaktivdir', 1;
        END

        SAVE TRANSACTION AfterCustomerValidation;
        PRINT 'Step 1: Müştəri validasiyası - OK';

        -- Step 2: Order yarat
        SET @StepNumber = 2;
        SET @SavePointName = 'AfterOrderCreation';

        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
SELECT @CustomerID, GETDATE(), SUM(Quantity * UnitPrice), 'Pending'
        FROM @Items;

        SET @OrderID = SCOPE_IDENTITY();
        SAVE TRANSACTION AfterOrderCreation;
        PRINT 'Step 2: Order yaradıldı - OK (ID: ' + CAST(@OrderID AS VARCHAR) + ')'
;

        -- Step 3: Order Details
        SET @StepNumber = 3;
        SET @SavePointName = 'AfterOrderDetails';
```

```
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        SELECT @OrderID, ProductID, Quantity, UnitPrice
        FROM @Items;

        SAVE TRANSACTION AfterOrderDetails;
        PRINT 'Step 3: Order Details əlavə edildi - OK';

        -- Step 4: Inventory yeniləməsi (kritik)
        SET @StepNumber = 4;
        SET @SavePointName = 'AfterInventoryUpdate';

        -- Stock yoxlanışı
        IF EXISTS (
            SELECT 1 FROM @Items i
            INNER JOIN Products p ON i.ProductID = p.ProductID
            WHERE p.StockQuantity < i.Quantity
        )
        BEGIN
            -- Inventory problemi - Order saxla amma pending status
            UPDATE Orders SET Status = 'Pending_Stock' WHERE OrderID = @OrderID;
PRINT 'Xəbərdarlıq: Stock kifayət etmir, order pending status-da saxlanı ldı';
COMMIT TRANSACTION ComplexOrder;
            RETURN;
        END

        -- Stock azaltma
        UPDATE p         SET p.StockQuantity =
p.StockQuantity - i.Quantity,
            p.LastUpdateDate = GETDATE()
FROM Products p
        INNER JOIN @Items i ON p.ProductID = i.ProductID;

        SAVE TRANSACTION AfterInventoryUpdate;
        PRINT 'Step 4: Inventory yeniləndi - OK';

        -- Step 5: Status yeniləməsi
        SET @StepNumber = 5;
        UPDATE Orders SET Status = 'Confirmed' WHERE OrderID = @OrderID;
        PRINT 'Step 5: Order confirmed - OK';

        -- Step 6: Audit log
        SET @StepNumber = 6;
        INSERT INTO OrderAuditLog (OrderID, Action, ActionDate, Details)
        VALUES (@OrderID, 'Order Created', GETDATE(),
                'Customer: ' + CAST(@CustomerID AS VARCHAR) + ', Items: ' +
                CAST((SELECT COUNT(*) FROM @Items) AS VARCHAR));

        PRINT 'Step 6: Audit log yazıldı - OK';

        -- Hamısı uğurlu - COMMIT
        IF @TransactionStarted = 1
        BEGIN
            COMMIT TRANSACTION ComplexOrder;
            PRINT '==================================';
            PRINT 'Transaksiya uğurla COMMIT olundu!';
            PRINT '==================================';
```

```
        END

    END TRY
    BEGIN CATCH
        -- Xəta baş verdi
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
        DECLARE @ErrorState INT = ERROR_STATE();

        PRINT '================================';
        PRINT 'XƏTA baş verdi!';
        PRINT 'Step: ' + CAST(@StepNumber AS VARCHAR);
        PRINT 'Mesaj: ' + @ErrorMessage;
        PRINT '================================';

        -- Transaksiya rollback
        IF @TransactionStarted = 1 AND @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION ComplexOrder;
            PRINT 'Transaksiya ROLLBACK edildi - bütün dəyişikliklər geri alındı';
        END

        -- Xətanı log-a yaz
        INSERT INTO ErrorLog (ErrorMessage, ErrorSeverity, ErrorState, ProcedureName
, ErrorLine)
        VALUES (@ErrorMessage, @ErrorSeverity, @ErrorState, 'ComplexOrderProcessing'
, ERROR_LINE());

        -- Xətanı yenidən qaldır
        THROW;
    END CATCH
END;
GO
```

3.3. Nested Transactions (İç-içə Transaksiyalar)

SQL Server fiziki olaraq nested (iç-içə) transaksiyaları dəstəkləmir. @@TRANCOUNT dəyişəni transaksiya səviyyəsini izləsə də, yalnız ən xarici COMMIT real commit-dir. İç transaksiyaların COMMIT-i yalnız @@TRANCOUNT-u azaldır. Ancaq ROLLBACK bütün səviyyələri geri alır!

```
-- Nested Transactions nümunəsi
CREATE PROCEDURE OuterProcedure
AS
BEGIN
    PRINT 'Xarici prosedur başladı';
    PRINT 'TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 0

    BEGIN TRANSACTION OuterTran;
    PRINT 'Xarici transaksiya başladı';
    PRINT 'TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 1

    INSERT INTO Log (Message) VALUES ('Xarici əməliyyat');

    -- Daxili proseduru çağır
    EXEC InnerProcedure;

    PRINT 'Xarici transaksiya commit olunur';     COMMIT
TRANSACTION OuterTran;  -- YALNIZmm bu real commit-dir!
    PRINT 'TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 0
END;
GO

CREATE PROCEDURE InnerProcedure
AS
BEGIN
    PRINT 'Daxili prosedur başladı';
    PRINT 'TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 1

    BEGIN TRANSACTION InnerTran;
    PRINT 'Daxili transaksiya başladı';
    PRINT 'TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 2

    INSERT INTO Log (Message) VALUES ('Daxili əməliyyat');
        COMMIT TRANSACTION InnerTran;  -- Yalnız TRANCOUNT
azalır!
    PRINT 'Daxili transaksiya "commit" olundu (əslində yox)';
PRINT 'TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 1
END;
GO

-- Test:
EXEC OuterProcedure;

-- VACİB: Nested Transactions ilə ROLLBACK
CREATE PROCEDURE NestedWithRollback
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION Outer;
        PRINT 'Outer TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 1
```

```
        INSERT INTO Accounts (AccountName, Balance) VALUES ('Test1', 1000);

        BEGIN TRANSACTION Inner;
        PRINT 'Inner TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  -- 2

        INSERT INTO Accounts (AccountName, Balance) VALUES ('Test2', 2000);

        -- Daxili transaksiyada xəta
        RAISERROR('Test xətası!', 16, 1);

        COMMIT TRANSACTION Inner;
        COMMIT TRANSACTION Outer;

    END TRY
    BEGIN CATCH
        PRINT 'Xəta: ' + ERROR_MESSAGE();

        IF @@TRANCOUNT > 0
        BEGIN
            PRINT 'ROLLBACK öncəsi TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);
ROLLBACK TRANSACTION;  -- BÜTÜN səviyyələri geri alır!
            PRINT 'ROLLBACK sonrası TRANCOUNT: ' + CAST(@@TRANCOUNT AS VARCHAR);  --
 0
        END

        PRINT 'HƏR İKİ insert geri alındı! (Test1 və Test2)';
    END CATCH
END;
GO

EXEC NestedWithRollback;

-- Düzgün pattern: SAVE TRANSACTION istifadə et
CREATE PROCEDURE ProperNestedPattern
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION MainTran;

        INSERT INTO Orders (OrderDate) VALUES (GETDATE());
        DECLARE @OrderID INT = SCOPE_IDENTITY();

        -- Savepoint yarat (nested transaksiya əvəzinə)
        SAVE TRANSACTION BeforeDetails;

        BEGIN TRY
            INSERT INTO OrderDetails (OrderID, ProductID, Quantity)
SELECT @OrderID, ProductID, Quantity FROM TempOrderItems;

            -- Validation
            IF (SELECT COUNT(*) FROM OrderDetails WHERE OrderID = @OrderID) = 0
            BEGIN
                -- Yalnız details-i geri al, Order-i saxla
                ROLLBACK TRANSACTION BeforeDetails;

                UPDATE Orders SET Status = 'Empty' WHERE OrderID = @OrderID;
            END
```

```
        END TRY
        BEGIN CATCH
            -- Details-də xəta - yalnız onu geri al
            ROLLBACK TRANSACTION BeforeDetails;
            PRINT 'Details geri alındı, amma Order qaldı';
        END CATCH

        COMMIT TRANSACTION MainTran;

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

3.4. Distributed Transactions

Distributed Transactions bir neçə verilənlər bazası və ya server arasında koordinasiya olunan transaksiyalardır. SQL Server Microsoft Distributed Transaction Coordinator (MSDTC) vasitəsilə distributed transactions-ı dəstəkləyir.

```
-- Distributed Transaction nümunəsi
-- Server1 və Server2 arasında transaksiya

BEGIN DISTRIBUTED TRANSACTION;
    -- Server1-də əməliyyat
    UPDATE Server1.Database1.dbo.Accounts
    SET Balance = Balance - 1000
    WHERE AccountID = 1;

    -- Server2-də əməliyyat
    UPDATE Server2.Database2.dbo.Accounts
    SET Balance = Balance + 1000
    WHERE AccountID = 2;

    -- Hər iki server-də uğurlu olarsa
COMMIT TRANSACTION;

-- Linked Server ilə distributed transaction
CREATE PROCEDURE CrossServerTransfer
    @FromServer VARCHAR(50),
    @ToServer VARCHAR(50),
    @FromAccount INT,
    @ToAccount INT,
    @Amount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @SQL1 NVARCHAR(MAX);
    DECLARE @SQL2 NVARCHAR(MAX);

    BEGIN TRY
```

```
        BEGIN DISTRIBUTED TRANSACTION;

        -- From server-də əməliyyat
        SET @SQL1 = 'UPDATE ' + QUOTENAME(@FromServer) + '.BankDB.dbo.Accounts ' +
'SET Balance = Balance - @Amount WHERE AccountID = @AccountID';

        EXEC sp_executesql @SQL1,
            N'@Amount DECIMAL(18,2), @AccountID INT',
            @Amount, @FromAccount;

        -- To server-də əməliyyat
        SET @SQL2 = 'UPDATE ' + QUOTENAME(@ToServer) + '.BankDB.dbo.Accounts ' +
'SET Balance = Balance + @Amount WHERE AccountID = @AccountID';

        EXEC sp_executesql @SQL2,
            N'@Amount DECIMAL(18,2), @AccountID INT',
            @Amount, @ToAccount;

        COMMIT TRANSACTION;
        PRINT 'Cross-server transfer uğurla tamamlandı';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

-- Linked Server yaratmaq (bir dəfəlik)
EXEC sp_addlinkedserver
    @server = 'RemoteServer',
    @srvproduct = '',
    @provider = 'SQLNCLI',
    @datasrc = 'RemoteServerName';

-- Distributed Transaction-ı monitor etmək
SELECT * FROM sys.dm_tran_active_transactions
WHERE transaction_type = 4;  -- 4 = Distributed transaction
```

3.5. Transaction Log

Transaction Log SQL Server-in ən vacib komponentlərindən biridir. Bütün transaksiyalar əvvəlcə log-a yazılır, sonra data file-a yazılır (Write-Ahead Logging WAL prinsipi). Bu, durability və recovery imkanlarını təmin edir.

```
-- Transaction Log-un izlənməsi və idarə edilməsi

-- 1. Transaction Log-un ölçüsü və istifadəsi
SELECT     DB_NAME(database_id) AS DatabaseName,     name AS
LogFileName,     type_desc AS FileType,     size * 8.0 / 1024 AS SizeMB,
CAST(FILEPROPERTY(name, 'SpaceUsed') AS INT) * 8.0 / 1024 AS UsedMB,
    size * 8.0 / 1024 - CAST(FILEPROPERTY(name, 'SpaceUsed') AS INT) * 8.0 / 1024 AS
FreeMB
FROM sys.master_files
WHERE type_desc = 'LOG'
ORDER BY DatabaseName;

-- 2. Aktiv transaksiyaları görmək
SELECT
    s.session_id,
    s.login_name,
    t.transaction_id,
    t.name AS TransactionName,
    t.transaction_begin_time,     DATEDIFF(SECOND, t.transaction_begin_time,
GETDATE()) AS Duration_Seconds,     t.transaction_type,     CASE
t.transaction_state
        WHEN 0 THEN 'Initializing'
        WHEN 1 THEN 'Initialized'
        WHEN 2 THEN 'Active'
        WHEN 3 THEN 'Ended'
        WHEN 4 THEN 'Committing'
        WHEN 5 THEN 'Prepared'
        WHEN 6 THEN 'Committed'
        WHEN 7 THEN 'Rolling back'
        WHEN 8 THEN 'Rolled back'
    END AS TransactionState
FROM sys.dm_tran_active_transactions t LEFT JOIN sys.dm_tran_session_transactions st
ON t.transaction_id = st.transaction_i d LEFT JOIN sys.dm_exec_sessions s ON
st.session_id = s.session_id
WHERE t.transaction_begin_time < DATEADD(MINUTE, -1, GETDATE())
ORDER BY t.transaction_begin_time;

-- 3. Transaction Log-u backup et
BACKUP LOG MyDatabase TO DISK = 'C:\Backup\MyDatabase_Log.trn'
WITH COMPRESSION, STATS = 10;

-- 4. Transaction Log-u shrink et (ehtiyatla!) --
Qeyd: Production-da shrink etmək tövsiyə olunmur!
USE MyDatabase;
GO

-- Əvvəlcə log backup et
```

```sql
BACKUP LOG MyDatabase TO DISK = 'C:\Backup\MyDatabase_Log.trn';

-- Sonra shrink et
DBCC SHRINKFILE (MyDatabase_Log, 100);  -- 100 MB-a qədər kiçilt

-- 5. Recovery Model-i dəyişdirmək
-- SIMPLE - Log avtomatik təmizlənir, point-in-time recovery yoxdur
ALTER DATABASE MyDatabase SET RECOVERY SIMPLE;

-- FULL - Bütün transaksiyalar log-a yazılır, point-in-time recovery var
ALTER DATABASE MyDatabase SET RECOVERY FULL;

-- BULK_LOGGED - Bulk əməliyyatlar minimal log-lanır
ALTER DATABASE MyDatabase SET RECOVERY BULK_LOGGED;

-- 6. Transaction Log-dakı məlumatları görmək (advanced)
SELECT
    [Current LSN],
    Operation,
    [Transaction ID],
    [Transaction Name],
    [Begin Time],
    [Transaction SID],
    AllocUnitName
FROM fn_dblog(NULL, NULL)
WHERE Operation IN ('LOP_BEGIN_XACT', 'LOP_COMMIT_XACT', 'LOP_ABORT_XACT')
ORDER BY [Begin Time] DESC;

-- 7. VLF (Virtual Log File) sayını yoxla
-- Çox VLF performansa mənfi təsir göstərir
DBCC LOGINFO;

-- 8. Long-running transactions-ı tap
SELECT
    s.session_id,
    s.login_name,
    t.transaction_begin_time,     DATEDIFF(MINUTE, t.transaction_begin_time,
GETDATE()) AS Duration_Minutes,     r.command,
    r.wait_type,
    r.wait_time,
    st.text AS QueryText FROM sys.dm_tran_active_transactions t INNER JOIN
sys.dm_tran_session_transactions ts ON t.transaction_id = ts.transaction_ id INNER
JOIN sys.dm_exec_sessions s ON ts.session_id = s.session_id
LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) st
WHERE DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) > 5  -- 5 dəqiqədən çox
ORDER BY Duration_Minutes DESC;
```

4. Rollback və Savepoint

4.1. ROLLBACK Mexanizmi

ROLLBACK əmri transaksiya daxilində edilmiş bütün dəyişiklikləri geri alır və verilənlər bazasını transaksiya başlamazdan əvvəlki vəziyyətə qaytarır. SQL Server bunu Transaction Log vasitəsilə edir.

```
-- ROLLBACK əsasları
BEGIN TRANSACTION;
    UPDATE Products SET Price = Price * 1.1;  -- 10% artım
    UPDATE Products SET StockQuantity = 0;     -- Bütün stock-u sıfırla

    SELECT COUNT(*) AS AffectedProducts FROM Products WHERE StockQuantity = 0;
        -- Nəticədən razı
deyilsənsə...
ROLLBACK TRANSACTION;

-- Nəticə: HEÇ BİR dəyişiklik qalmadı!
SELECT COUNT(*) AS AffectedProducts FROM Products WHERE StockQuantity = 0;  -- 0

-- Şərti ROLLBACK
CREATE PROCEDURE ConditionalUpdate
    @TargetCount INT,
    @NewPrice DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRANSACTION;
        UPDATE Products
        SET Price = @NewPrice
        WHERE CategoryID = 5;

        DECLARE @AffectedRows INT = @@ROWCOUNT;

        IF @AffectedRows > @TargetCount
        BEGIN             -- Çox sətr
təsirləndi - təhlükəli!
            ROLLBACK TRANSACTION;
            RAISERROR('Çox sətr təsirlənəcək (%d > %d), rollback edildi!', 16, 1,
                      @AffectedRows, @TargetCount);
            RETURN -1;
        END

    COMMIT TRANSACTION;
    PRINT 'Update uğurla tamamlandı: ' + CAST(@AffectedRows AS VARCHAR) + ' sətr';
    RETURN 0;
END;
GO

-- Avtomatik ROLLBACK (SET XACT_ABORT ON)
SET XACT_ABORT ON;  -- Hər hansı xətada avtomatik rollback

BEGIN TRANSACTION;
    INSERT INTO Orders (CustomerID, OrderDate) VALUES (1, GETDATE());
```

```
    -- Bu sətr xəta verəcək (məsələn, FK constraint)
    INSERT INTO OrderDetails (OrderID, ProductID, Quantity)
    VALUES (99999, 1, 10);  -- OrderID mövcud deyil

    -- Bu sətrə heç vaxt çatmaz, çünki avtomatik rollback olur
COMMIT TRANSACTION;

SET XACT_ABORT OFF;

-- Manual error handling ilə ROLLBACK
CREATE PROCEDURE SafeOrderProcessing
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Əməliyyat 1
        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount)
        VALUES (1001, GETDATE(), 500);

        DECLARE @OrderID INT = SCOPE_IDENTITY();

        -- Əməliyyat 2 (məsələn, xəta verəcək)
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
VALUES (@OrderID, 9999, 10, 50);  -- ProductID mövcud deyil (FK error)

        -- Əməliyyat 3
        UPDATE Customers SET TotalOrders = TotalOrders + 1 WHERE CustomerID = 1001;

        COMMIT TRANSACTION;
        PRINT 'Uğurlu!';

    END TRY
    BEGIN CATCH
        -- Xəta baş verdi - ROLLBACK
        IF @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION;
            PRINT 'ROLLBACK: ' + ERROR_MESSAGE();
        END

        -- Xətanı yenidən qaldır
        THROW;
    END CATCH
END;
GO

EXEC SafeOrderProcessing;
```

4.2. Savepoint Strategiyaları

SAVEPOINT transaksiya daxilində aralıq nöqtələr yaratmağa imkan verir. Bu, bütün transaksiyanı deyil, yalnız müəyyən hissəsini geri almağa kömək edir. SQL Server-də SAVE TRANSACTION əmri ilə istifadə olunur.

```
-- Savepoint əsasları
BEGIN TRANSACTION;

    -- Əməliyyat 1
    INSERT INTO Customers (CustomerName, Email) VALUES ('Ali Məmmədov', 'ali@example
.com');
    DECLARE @CustomerID INT = SCOPE_IDENTITY();

    -- Savepoint 1
    SAVE TRANSACTION SavePoint1;
    PRINT 'SavePoint1 yaradıldı';

    -- Əməliyyat 2
    INSERT INTO Orders (CustomerID, OrderDate, TotalAmount)
    VALUES (@CustomerID, GETDATE(), 1000);
    DECLARE @OrderID INT = SCOPE_IDENTITY();

    -- Savepoint 2
    SAVE TRANSACTION SavePoint2;
    PRINT 'SavePoint2 yaradıldı';

    -- Əməliyyat 3
    INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
    VALUES (@OrderID, 101, 5, 200);
        -- Yoxlama: əgər məbləğ
səhvdirsə...
    IF (SELECT SUM(Quantity * UnitPrice) FROM OrderDetails WHERE OrderID = @OrderID)
 <> 1000
    BEGIN
        -- Yalnız OrderDetails-i geri al
        ROLLBACK TRANSACTION SavePoint2;
        PRINT 'OrderDetails geri alındı, amma Customer və Order saxlanıldı';

        -- Düzgün OrderDetails daxil et
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        VALUES (@OrderID, 101, 10, 100);  -- Düzgün hesablama
    END

COMMIT TRANSACTION;

-- Kompleks Savepoint Nümunəsi
CREATE PROCEDURE ProcessComplexOrder
    @CustomerID INT,
    @Items dbo.OrderItemType READONLY,
    @ApplyDiscount BIT = 0,
    @DiscountCode VARCHAR(50) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @OrderID INT;
    DECLARE @TotalAmount DECIMAL(18,2);
```

```
    DECLARE @DiscountAmount DECIMAL(18,2) = 0;

    BEGIN TRY
        BEGIN TRANSACTION ComplexOrder;

        -- Step 1: Order yaradılması
        SELECT @TotalAmount = SUM(Quantity * UnitPrice) FROM @Items;

        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
        VALUES (@CustomerID, GETDATE(), @TotalAmount, 'Pending');

        SET @OrderID = SCOPE_IDENTITY();
        SAVE TRANSACTION AfterOrderCreation;
        PRINT 'Order yaradıldı (ID: ' + CAST(@OrderID AS VARCHAR) + ')';

        -- Step 2: OrderDetails əlavə et
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        SELECT @OrderID, ProductID, Quantity, UnitPrice FROM @Items;

        SAVE TRANSACTION AfterOrderDetails;
        PRINT 'OrderDetails əlavə edildi';

        -- Step 3: Endirim tətbiq et (optional)
        IF @ApplyDiscount = 1 AND @DiscountCode IS NOT NULL
        BEGIN
            SAVE TRANSACTION BeforeDiscount;

            BEGIN TRY
                -- Endirim kodunu yoxla
                SELECT @DiscountAmount = (@TotalAmount * DiscountPercent / 100)
                FROM DiscountCodes
                WHERE Code = @DiscountCode
                  AND IsActive = 1
                  AND ValidFrom <= GETDATE()
                  AND ValidTo >= GETDATE();

                IF @DiscountAmount IS NULL OR @DiscountAmount = 0
                BEGIN
                    -- Endirim etibarsızdır - geri al
                    ROLLBACK TRANSACTION BeforeDiscount;
                    PRINT 'Endirim kodu etibarsızdır, tətbiq olunmadı';
                END
                ELSE
                BEGIN
                    -- Endirim tətbiq et
                    UPDATE Orders
                    SET DiscountAmount = @DiscountAmount,
                        FinalAmount = @TotalAmount - @DiscountAmount
                    WHERE OrderID = @OrderID;

                    PRINT 'Endirim tətbiq olundu: ' + CAST(@DiscountAmount AS VARCHA
R);
END

            END TRY
            BEGIN CATCH
                -- Endirim əməliyyatında xəta - yalnız endirim hissəsini geri al
                ROLLBACK TRANSACTION BeforeDiscount;
```

```
                PRINT 'Endirim xətası, amma Order saxlanıldı: ' + ERROR_MESSAGE();
            END CATCH
        END

        -- Step 4: Stock yeniləməsi (kritik)
        SAVE TRANSACTION BeforeStockUpdate;

        BEGIN TRY
            UPDATE p
            SET p.StockQuantity = p.StockQuantity - i.Quantity
            FROM Products p
            INNER JOIN @Items i ON p.ProductID = i.ProductID;
                        -- Stock
mənfi oldu mu?
            IF EXISTS (SELECT 1 FROM Products WHERE StockQuantity < 0)
            BEGIN
                -- Stock-u geri al
                ROLLBACK TRANSACTION BeforeStockUpdate;

                -- Order-u pending status-da saxla
                UPDATE Orders SET Status = 'Pending_Stock' WHERE OrderID = @OrderID;
PRINT 'Stock kifayət etmir, Order pending status-da saxlanıldı';
            END
            ELSE
            BEGIN
                -- Hamısı OK - Order confirm et
                UPDATE Orders SET Status = 'Confirmed' WHERE OrderID = @OrderID;
                PRINT 'Stock yeniləndi, Order confirmed';
            END

        END TRY
        BEGIN CATCH
            -- Stock update xətası - geri al amma Order saxla
            ROLLBACK TRANSACTION BeforeStockUpdate;
            UPDATE Orders SET Status = 'Pending_Stock', Notes = ERROR_MESSAGE() WHER
E OrderID = @OrderID;
            PRINT 'Stock update xətası: ' + ERROR_MESSAGE();
        END CATCH

        -- Əsas transaksiyanı commit et
        COMMIT TRANSACTION ComplexOrder;
        PRINT '==================================';
        PRINT 'Əməliyyat tamamlandı!';
        PRINT 'Order ID: ' + CAST(@OrderID AS VARCHAR);
        PRINT '==================================';

    END TRY
    BEGIN CATCH
        -- Kritik xəta - bütün transaksiyanı geri al
        IF @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION ComplexOrder;
            PRINT 'Kritik xəta - bütün əməliyyat geri alındı!';
        END
        THROW;
    END CATCH
END;
GO
```

Yekun Xülasə və Praktik Tövsiyələr

Bu ətraflı təhsil materialında Microsoft SQL Server-in əsas konsepsiyalarını, ACID prinsiplərini, transaksiyaları, sessiyaları, connection pooling və isolation səviyyələrini dərindən öyrəndik. Əsas Nöqtələr: • Stored Procedures - kod təşkili və performans üçün • ACID - məlumat bütövlüyünün 4 prinsipi • Transactions - kompleks əməliyyatların idarəsi • Rollback/Savepoint - dəyişikliklərin idarəsi • Sessions connection idarəetməsi • Connection Pooling - performans optimallaşdırması • Isolation Levels - paralel əməliyyatların idarəsi Praktikada tətbiq edin və öyrənməyə davam edin!